Service Discovery

how services find each other when addresses keep changing

Watch the 4-min video

In one line: service discovery keeps a live registry of which instances exist and where, so callers can find them by name; in client-side discovery the caller looks up and balances itself, in server-side discovery a load balancer does it, DNS is the simplest form, and a service mesh combines both.

1 Addresses that never sit still

Your order service needs to call the payment service. But payments runs as twelve containers, their IP addresses change on every deploy, and autoscaling adds and removes them all day. Hard-coding addresses is impossible. Services need a way to find each other by **name**: **service discovery**.

2 The service registry

At the centre is a **service registry**: a live directory of which instances exist and where. Instances **register** when they start (name, address, port, maybe a version tag) and **deregister** when they stop. Health checks or heartbeats with a TTL remove the ones that die silently.

Common registries: **Consul**, **etcd**, **ZooKeeper** and Netflix **Eureka**. In Kubernetes, the API server and its **EndpointSlices** play this role, updated automatically as pods become ready or go away.

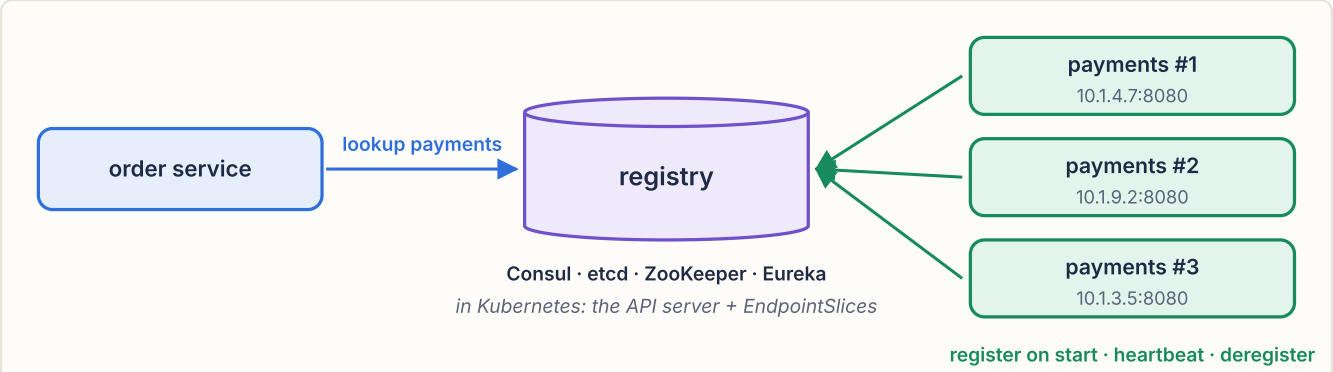


Figure 1. Instances register and send heartbeats; callers ask the registry where a service lives.

3 Client-side and server-side discovery

In **client-side discovery** the calling service queries the registry itself, gets the list of healthy instances, and picks one with its own load balancing, usually through a library. Netflix's Eureka with the Ribbon client worked this way. It's efficient (no extra hop) and allows smart algorithms such as least requests, but every client language needs the library, and discovery logic spreads into your code.

In **server-side discovery** the client calls a fixed address, a load balancer or router, which queries the registry and forwards the request. AWS load balancers with target groups work this way. Clients stay simple and language-independent, but you add a network hop, and the router must be highly available.

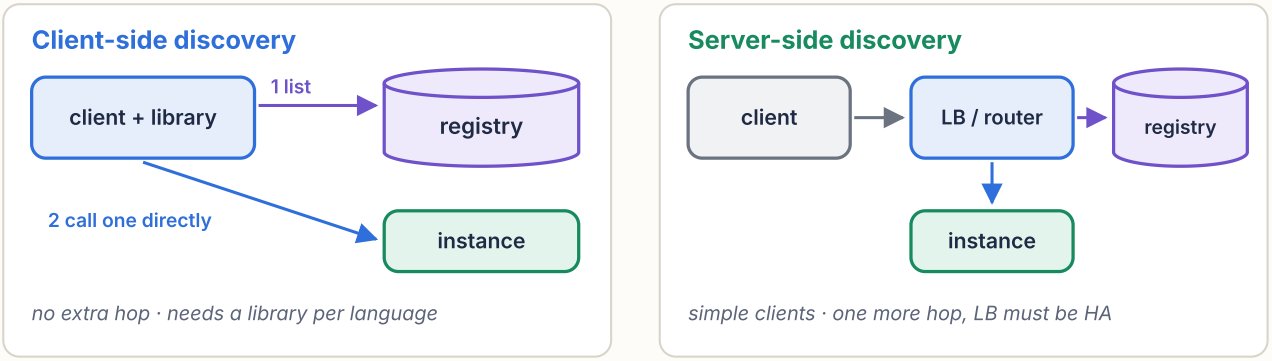


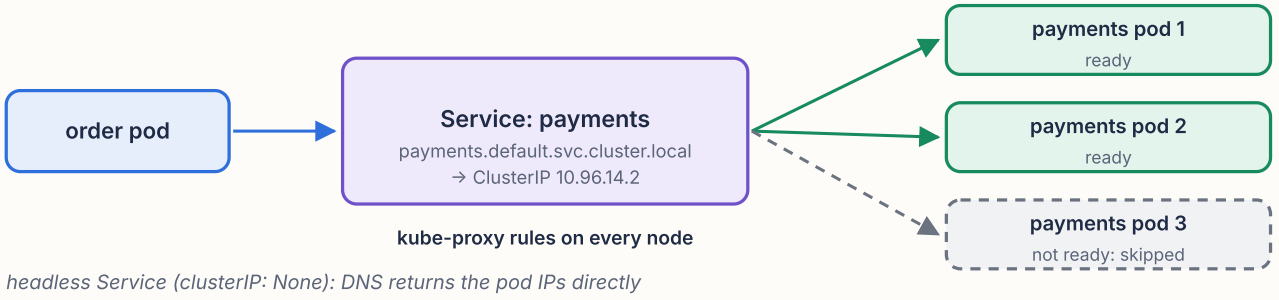
Figure 2. Client-side: the caller asks the registry and calls an instance directly. Server-side: the caller hits a router that does the lookup.

4 DNS-based discovery

The simplest form: a service name resolves to the IP addresses of its instances, and **SRV records** can include ports too. Every language supports DNS, so there is nothing to install. The catch is **caching**: clients and resolvers may keep old answers until the TTL expires (and some runtimes cache longer than the TTL), so removed instances can keep receiving traffic for a while. Keep TTLs short (5 to 30 seconds) and still handle connection errors with retries.

WORKED EXAMPLE: A KUBERNETES SERVICE

You create a Service named `payments`. Cluster DNS gives it the name `payments.default.svc.cluster.local`, which resolves to a stable virtual IP (the ClusterIP). **kube-proxy** programs rules (iptables or IPVS) on every node, so traffic to that IP is spread across the **ready** pods only. Pods come and go; the name and the IP stay the same. A **headless** Service (`clusterIP: None`) skips the virtual IP, and DNS returns the pod IPs directly, which suits clients that balance themselves, such as gRPC.



headless Service (clusterIP: None): DNS returns the pod IPs directly

Figure 3. A Kubernetes Service: a stable name and virtual IP in front of the ready pods; a pod that is not ready gets no traffic.

5 Service mesh: both at once

Service meshes such as **Istio** and **Linkerd** combine the styles. A sidecar proxy next to each service gets the endpoint list from the mesh's control plane and balances load locally. Your code just calls a name, as if it were server-side, but the work happens client-side, in the sidecar, with retries, timeouts and mutual TLS included.

COMMON MISTAKES

- Hard-coding IPs or long-lived DNS caches (the JVM's old default cached forever), so callers keep hitting dead instances.
- Instances that crash without deregistering, and no heartbeat TTL to clean them up.
- A single registry node: the registry is critical infrastructure and must be replicated (Consul and etcd run 3 or 5 nodes with Raft).
- Gaps during deploys: a new instance registered before it's ready, or an old one removed while requests are in flight.

6 Key terms

Service registry: live directory of instances and addresses

Registration: an instance announces itself on start

Heartbeat / TTL: proof of life; silent instances expire

Client-side discovery: caller looks up and load balances itself

Server-side discovery: a router looks up and forwards

SRV record: DNS record with host and port

ClusterIP: a Kubernetes Service's stable virtual IP

Headless Service: DNS returns pod IPs, no virtual IP

7 Interview questions

Q1. Compare client-side and server-side discovery. BASIC

In client-side discovery the caller queries the registry and load balances itself, avoiding an extra hop but needing a library in every language; in server-side discovery the caller hits a load balancer or router that does the lookup, keeping clients simple but adding a hop.

Q2. What does a Kubernetes Service give you? BASIC

A stable DNS name and virtual IP that kube-proxy spreads across ready pods: server-side discovery built into the platform.

Q3. What are the downsides of DNS-based discovery? BASIC

Caching: clients and resolvers may use stale answers until the TTL expires, so removed instances still get traffic; DNS also gives no load information and only coarse balancing.

Q4. After a deploy, some requests still go to old, terminated instances for a minute. Why? SCENARIO

Callers cached DNS or a stale registry list. Shorten TTLs, make sure instances deregister before shutting down, drain connections, and retry on connection errors.

Q5. How does a gRPC client balance across pods in Kubernetes? SCENARIO

A normal Service pins its HTTP/2 connection to one pod. Use a headless Service so DNS returns all pod IPs and the gRPC client balances per request, or use a mesh sidecar.

Q6. What happens if the registry goes down? TRICKY

New instances can't be found, but well-built clients keep using their last known list. Run the registry replicated (Raft quorum of 3 or 5) and let clients cache results so a registry outage doesn't become a service outage.

Next: SD-G-009 · Reverse Proxy Configuration backendroadmap.com